

VPS Hosting

Virtual Private Server / (cloud server)

React `npm create-react-app foldername`

Vite `npm create vite@latest`

Components & Props

Basic syntax:-

(Components / navbar.jsx)

```
import React from "react"
import "./navbar.css" // for styling
```

```
const navbar = (props) => {
  return (
    <nav>
      <h1> { props.heading } </h1>
    </nav>
  )
}
```

Object.
↳ props (dynamic variables which can be added)
only one tag can be used in return.

export default ~~const~~ navbar.

@ App.jsx

```
import Navbar from "../components/Navbar"
```

```
function App() {
```

```
  return (
```

```
    <div>
```

```
      <Navbar heading="Abc" />
```

```
    </div>
```

```
  )
```

```
}
```

```
export default App;
```

#

HOOKS

functions which are used to use state & other react features ~~in~~ directly inside functional components.

Syntax: (example)

```
const a = useState(0);
```

Initialization value

↓
Dynamic variable
which can be
used directly

↘ built-in Hook

```
const [a, setCount] = useState(0);
```


useEffect() (Used to make side effects)

Syntax:-

`useEffect(() => {`

`// something you want to do.`

`}, [])`

array (contains things on change of which it has to trigger the ~~the~~ `useEffect`)

Cases:-

① Run on every render.

`useEffect(() => { // code }, )`

no array

only.
② Run ~~on~~ first render.

`useEffect(() => { // code }, [])`

empty array

③ Run when any state or variable is changed.

`useEffect(() => { // code }, [a, b, c])`

when these variables are modified, `useEffect` will be triggered

Clean Up function. (its optional)
called when the component is unmounts

~~useEffect (c) => {
 // code
}, [])~~

useEffect (c) => {

// Code

return c => {

// clean-up function

}
}, [])

~~useRef~~ useRef (c)
used to persist the Value of any
variable across renders

Syntax :-

~~const a = useRef~~

const a = useRef (c)

to access 'a' :-

a.current

Note:-

★ → when `useEffect` is used when Component is re-rendered.

★ → When `useRef` is used then Component is NOT re-rendered.

Conditional Rendering

```
const [showbtn, setshowbtn] = useState(true);
```

```
{ showbtn ? <button> show-btn </button> : "" }
```

or

```
{ showbtn && <button> show-btn </button> }
```

↳ state.

To show or hide anything based on a state.

Displaying multiple ~~obj~~ ~~items~~ ~~objects~~ ~~using~~ components using a loop for a list.

`todos` → Array

→ Props

`todos.map (todo => {`

return `<Todo key={todo.title} todo={todo} />`

`})}`

↓
Component

↓
Unique Key must be given.



```
1  import { useState } from 'react'
2  import './App.css'
3
4  function App() {
5    const [val, setval] = useState({});
6    function input_handler(e) {
7      setval({ ...val, [e.target.name]: e.target.value })
8    }
9    return (
10      <section className='flex-col'>
11        <div className="flex-col">
12          <label>Fname: </label>
13          <input name='fname' onChange={input_handler} type="text" />
14        </div>
15        <div className="flex-col mt-5">
16          <label>Lname: </label>
17          <input name='lname' onChange={input_handler} type="text" />
18        </div>
19        <p className='flex p-2'>Name: {val.fname} {val.lname}</p>
20        {console.log(val)}
21      </section>
22    )
23  }
24
25  export default App
```

// now we can use multiple React Router.

npm i react-router-dom.

import {createBrowserRouter} from 'react-router-dom'
Router Provide.

~~Then~~

// Then inside App function:-

```
const router = createBrowserRouter([
```

```
  {
```

```
    path: "/",
```

```
    element: <Home />
```

```
  },
```

```
  {
```

```
    path: "/about",
```

```
    element: <About />
```

```
  }
])
```

// endpoint

// react component
to display @
that endpoint

```
<RouterProvider router={router} />
```




router - App.jsx

```
1  import { createBrowserRouter, RouterProvider } from 'react-router-dom'
2  import Home from './components/home'
3  import About from './components/about'
4  import Policy from './components/policy'
5  import Tnc from './components/tnc'
6  function App() {
7    const router = createBrowserRouter([
8      {
9        path: "/",
10       element: <Home />
11     },
12     {
13       path: "/about",
14       element: <About />
15     },
16     {
17       path: "/policy",
18       element: <Policy />
19     },
20     {
21       path: "/tnc",
22       element: <Tnc />
23     },
24   ])
25   return (
26     <>
27       <RouterProvider router={router} />
28     </>
29   )
30 }
31 export default App
```



router - header.jsx

```
1  import React from 'react'
2  import { Link } from 'react-router-dom'
3  function header() {
4    return (
5      <header>
6        <Link to="/">Home</Link>{/*similar to <a href='/about'><a/>*/}
7        <Link to="/about">About</Link>
8        <Link to="/">Contact</Link>
9      </header>
10    )
11  }
12  export default header
```

Dynamic Routing

```
{ path: "/user/:username",
  element: <User />
}
```

→ This is Dynamic variable.
like.
@ user/.

user/abc
↳ anything.

then inside user.jsx

```
import { useParams } from 'react-router-dom'
```

```
function user() {
```

```
  const params = useParams();
```

↳ now this is the obj. containing all variables.

```
  <h1> i am {params.username} </h1>
```

```
}
```

```
export default user;
```




```
1 import { createBrowserRouter, RouterProvider } from 'react-router-dom'
2 import User from './components/user'
3 function App() {
4   const router = createBrowserRouter([
5     {
6       path: "/user/:username",
7       element: <User />
8     }
9   ])
10  return (
11    <>
12      <RouterProvider router={router} />
13    </>
14  )
15 }
16 export default App
```



```
1 import Header from './components/header'
2 import Footer from './components/footer'
3 import { useParams } from 'react-router-dom'
4 function user() {
5   const Params = useParams()
6   return (
7     <>
8       <Header />
9       <h1 className='text-3xl'>i am {Params.username}</h1>
10      <Footer />
11    </>
12  )
13 }
14 export default user
```

NavLink (import {NavLink} from 'react-router-dom')
to highlight ~~the~~ the active content (tab) in
navbar.

Like:-

[Home](#)! [About](#) [Contact](#)

This is flower.

Footer

Syntax:-

```
<NavLink classname={e => e.isActive ? "text-red-500" : ""}
to="/">Home</NavLink>
```

```

1 import { NavLink } from 'react-router-dom'
2 function header() {
3   return (
4     <header>
5       <NavLink className={e => { return e.isActive ? "text-red-500" : "" }} to="/">Home</NavLink>
6       <NavLink className={e => { return e.isActive ? "text-red-500" : "" }} to="/about">About</NavLink>
7       <NavLink className={e => { return e.isActive ? "text-red-500" : "" }} to="/tnc">T&C</NavLink>
8     </header>
9   )
10 }
11 export default header

```


useContent (Content API)

used to Communicate variables & states across the components directly.

in content.js (not .jsx)

```
import {createContent} from "react";
```

```
export const countercontent = createContent(0);
```

then import it in app.jsx and wrap the component in `<countercontent.Provider>` and provide value in it.

like :-

in App.jsx

```
import {countercontent} from 'respective-file'
```

```
return (
```

```
<>
```

```
<countercontent.Provider value={any value}>
```

```
{/* code */}
```

```
</countercontent.Provider>
```

```
)
```

```
}
```



usecontext - App.jsx

```
1 import React, { useContext, useState } from 'react'
2 import { createBrowserRouter, RouterProvider } from 'react-router-dom'
3 import Home from './components/home'
4 function App() {
5   const [Count, setCount] = useState(0)
6   return (
7     <>
8       <countercontext.Provider value={{ Count, setCount }}>
9         <Home />
10      </countercontext.Provider>
11    </>
12  )
13 }
14 export default App
```



usecontext - context.js

```
1 import { createContext } from "react";
2 export const countercontext = createContext(0);
```



usecontext - home.jsx

```
1 import React from 'react'
2 import { useContext } from 'react'
3 import { countercontext } from '../context/context.js'
4 function home() {
5   const counter = useContext(countercontext)
6   return (
7     <button onClick={() => {counter.setCount(counter.Count+1)}}>
8       Counter: {counter.Count}
9     </button>
10  )
11 }
12
13 export default home
```


///

useMemo

- to memoize something (variable)
- i.e. to prevent any specific function or a statement to rerun when the component is rerendered to optimize the Frontend
- Can be used ~~to prevent~~ to prevent resource intensive tasks to rerun every time the component renders.

import {useMemo} from 'react'

```
const abc = useMemo(() => {
  /* any function expensive computation */
}, [dependency array])
```

Specific things on which change the fn inside useMemo ~~not~~ reruns.

///

memo

in any Component it prevent rerender until its props are changed.

```
import React, {memo} from 'react'
function Navbar (props) {
  /* code */
  return ( /* component */ )
}
```

export default memo(Navbar)

useCallback() [import {useCallback} from 'react']

→ to memoize function.

→ To make a function re initialize or rerun when some specific thing is changed otherwise freeze the function.

Syntax:-

any function on which u want to apply this

const any_function = useCallback([] => { }, [])

any fn

dependency array

Handling forms + Connecting to Express backend using React Hook form

npm i react-hook-form

import {useForm} from "react-hook-form"

Using ... register function in inputs.

<input { ... register("username", { validations })

name of the I/P

various types of validations

add disabled = { isSubmitting } in button to disable it while submission is in process to avoid chaos.

<button disabled = { isSubmitting } type = "submit"> submit
</button>

Handling Serverside Errors.

Custom errors

Step 1) add 'setError' in useForm obj.

↓
This can be anything we'll be setting it later.

~~Step 2~~ Step 2) ~~add~~ define setError in on Submit fun.

Syntax :-

setError ("anyKey", {message: "err msg"})

Step 3) use anyKey where you want to display error

Syntax :-

{errors.anyKey && <p>{errors.anyKey.message} </p>}


```

1  import { useState } from 'react'
2  import { useForm } from 'react-hook-form'
3  function App() {
4    const {
5      register,
6      handleSubmit,
7      watch,
8      formState: { errors },
9    } = useForm()
10   const onSubmit = data => { console.log(data) }
11   return (
12     <>
13       <h1 className='text-3xl'>Form</h1>
14       <form onSubmit={handleSubmit(onSubmit)}>
15         <input type="email" placeholder='Email' {...register("username")} />
16         <input type="password" placeholder='password' {...register("password")} />
17         <button type="submit">Submit</button>
18       </form>
19     </>
20   )
21 }
22 export default App

```

```

1  import { useState } from 'react'
2  import { useForm } from 'react-hook-form'
3  function App() {
4    const {
5      register,
6      handleSubmit,
7      watch,
8      formState: { errors },
9    } = useForm()
10   const onSubmit = data => { console.log(data) }
11   return (
12     <>
13       <h1 className='text-3xl'>Form</h1>
14       <form onSubmit={handleSubmit(onSubmit)}>
15
16         <input type="text" placeholder='Email' {...register("username", {
17           required: true,
18           minLength: { value: 3, message: `value should not below 3` }
19         })} />
20
21         <input type="password" placeholder='password' {...register("password")} />
22
23         {errors.username ? <p className="text-red-500">{errors.username.message}</p> : ''}
24
25         <button type="submit">Submit</button>
26       </form>
27     </>
28   )
29 }
30 export default App

```


Redux

(~~Redux~~ store.

a single source of truth)

use ~~documentat~~ documentation for further details

Next JS

`npm create-next-app@latest`

File based Routing

app.

`└ contact/page.js`
`└ about/page.js`
`└ page.js`

} Routes and Components.

by default Everything is a server Component.

to make any 'component' client component just add this at top of the component file.

↖
"use client";

#

Link Component

Instead of anchor `<a>` use `<Link>` to avoid reload.

```
import Link from 'next/link'
```

```
<Link href='/route'>Button</Link>
```

~~Meta~~

#

meta data (for SEO)

```
export const metadata = {
  title: "any title";
  description: "desc";
};
```

~~can~~ used inside `layout.js` and can be used inside any component

#

Script tag [used to inject JS]

```
import Script from 'next/script'
```

```
<Script>
```

```
</Script>
```


#

Image Component
used to optimize Images.

```
import Image from 'next/image';
```

```
<Image
```

```
  src = "source"
```

```
  width = {100}
```

```
  height = {100}
```

```
  alt = " "
```

} automatically adjusts
aspect ratio

#

NextJS API (Handling API Requests @Server)
file based endpoints

app/api/add/route.js

```
import { NextResponse } from 'next/server';
```

```
export async function POST (request) {  
  return NextResponse.json ({
```

```
    success: true,
```

```
    data: 'abc' } );
```

```
}
```

using it @ frontend.

```
let a = await fetch ("/api/add").
```

Server Actions in Next JS

@client side :-

```
"use client"
import {handleForm} from "../actions/form"
export default function Home() {
  return (
    <form action={ () => {handleForm} } >
      //code
    </form>
  )
}
```

@server side:-

```
"use server"
export const handleForm = async e => {
  //code
  console.log(e?.get(" " " "))
}
```

↓
name ~~attribute~~
attribute
of inputs -



```
1  "use server"
2  export const handleForm = async (e)=>{
3      console.log(e.get("name"), e.get("email"))
4  }
```



```
1  "use client";
2  import { handleForm } from "../actions/from";
3  export default function Home() {
4      return (
5          <main>
6              <form action={()=>{handleForm}}>
7                  <input type="email" name="email" id="email" placeholder="email" />
8                  <input type="name" name="name" id="name" placeholder="name" />
9              </form>
10         </main>
11     );
12 }
```

Middleware in Next.js

middleware.js

```
import { NextResponse } from 'next/server'
```

```
export function middleware (request) {  
  if (request.nextUrl.pathname.startsWith('/route'))  
  {  
    //code  
  }  
}
```

}

~~used to match URLs~~

```
export const config = {  
  matcher: '/about/:path*',  
}
```

Auth.js (used to authenticate the app)

for example -

"Sign in with Google"

Dynamic Routing

implemented using file based Routing
→ this can be used in Page.js

app/route/[slug]/page.js

This is end point.

params.slug

Example .

```
export default function Page ({params}) {  
  return <div> {params.slug} </div>  
}
```

Custom 404 Page.

app/not-found.jsx

// page for 404 not found Page.

for multiple parameters in slug

app/route/[...val]/page.js

```
export default function Page ({params}) {  
  console.log(params);  
  return <> something </>  
}
```

@ console:- for EP router array of /slug
{val: [array of , slug]}

// Layouts

Made inside layout.js can be made in route folders

Made to define a basic layout of the similar types of pages.

For Example:-

Header & Footer is required in every page so it should be defined inside root layout

grouping pages with same layout but not same parent End Point.

Example

app.
↳ (admin) ↗ apply paranthesis to skip any EP.

↳ login app / page.jsx	// ✓ login	X admin/login
↳ Signup / page.jsx		
↳ layout.jsx		

Navigation

usePathname (It's a hook so it should be used inside ^{any component})
used to get exact current path or
(link ~~to including~~ ~~end p~~ current End Point ~~of~~
~~Production~~) route

Syntax:-

```
import {usePathname} from next/navigation
export default function Navbar() {
  const path = usePathname();
}
```

useParams
used to get ~~link~~ End point params
like /slug ~~are~~ ~~!params~~

Syntax:-

```
import {useParams} from next/navigation
export default function Navbar() {
  const slug =
  const params = useParams()
  console.log(params.slug);
}
```

useRouter.

to navigate b/w end points with ease.

Syntax :-

```
import {useRouter} from 'next/navigation'
```

//inside component

```
const router = useRouter()
```

```
<button onClick={ () => { router.push('/any/route') } }  
  Btn.
```

```
</button>
```

This also can be done by `<Link>` but above method can also be used with `useEffect` & `setInterval` / `setTimeout` or any automatic navigation logic unlike `<Link>`.

useSearchParams.

used to get & data from ?query.

for ?blog=abc & name=Atharva.

```
import {useSearchParams} from 'next/navigation'
```

~~const useSea~~

```
const par = useSearchParams()
```

```
console.log ( par.blog par.get('blog'),  
              par.get('name') )
```


Data Fetching in NextJS

by default fetch function in nextJS ~~do~~ caches the data and not calls fetch frequently.

to force it to fetch everytime apply `{cache: 'no-store'}` in it

`fetch('link', {cache: 'no-store'})`

time based Revalidation.

`fetch('https://...', {next: {revalidate: 3600}})`
↓
1hr.

to revalidate data at a timed interval you can use the `next-revalidate` option of fetch to set the cache lifetime of a resource (in sec)

However there are some exceptions, fetch requests are not cached when.

- used inside a server Action
- Used inside a Route Handler that uses the POST method.

~~required~~
use :-

`export const dogDynamic = 'force-dynamic'`
to remove fetch data caching intentionally.

Environment Variables.

by default all Env. Variables are not as accessible @ client side but. if ~~we~~ ~~NEXT-P~~ ~~NEXT_PUBLIC~~ is applied then it will be accessible @ client side.

ID = abc. // not accessible @ client side

NEXT_PUBLIC_ID = def. // accessible @ client side.

• env.local is loaded & prioritised b4 .env.
• as ~~these~~ sometime. Production & local variables are different.

Styling in NextJS

CSS modules.

@ Home.module.css.

```
.red {  
  color: 'red';  
}
```

in component :-

```
import style from './styles/Home.module.css'
```

```
<div className={style.red} >  
  code ,  
</div>
```


`<style jsx>`
to make styles highly localized.

~~`<div>`~~

`<div className = 'abc'>`

code.

~~`<style jsx>`~~

some styles these styles only
applied in
div=abc and
not outside it

`</style>`

`</div>`

to make it
global; use
`<style jsx global>`